

```

import anthropic
import os
import httpx
import base64
from pdf_loader import PDFLoader

ANTHROPIC_API_KEY = os.getenv("ANTHROPIC_API_KEY", "YOUR_ANTHROPIC_API_KEY")

SYSTEM_PROMPT = """Sen O'zbekiston maktab biologiya fanining AI yordamchisissan.
5-sinf dan 11-sinf gacha bo'lgan biologiya darsliklarini yaxshi bilasan.

Qoidalar:
1. Faqat biologiya faniga oid savollarga javob ber
2. O'zbek tilida javob ber
3. Imkon qadar qisqa va aniq javob ber
4. Agar sinf ko'rsatilgan bo'lsa, o'sha sinf dasturiga mos javob ber
5. Test savolida to'g'ri javobni aniq ko'rsat (A, B, C, D)
6. Rasmlı testlarda rasmni diqqat bilan tahlil qil
7. Javobni HTML formatida ber (bold, italic ishlatısa bo'ladi)

Sinflar bo'yicha mavzular:
- 5-sinf: Tirik organizmlar, o'simliklar, hayvonlar (boshlang'ich)
- 6-sinf: O'simliklar biologiyasi
- 7-sinf: Hayvonlar biologiyasi
- 8-sinf: Inson anatomiyasi va fiziologiyasi
- 9-sinf: Umumiy biologiya (hujayra, genetika)
- 10-sinf: Genetika, evolyutsiya
- 11-sinf: Ekologiya, biotexnologiya
"""

class ClaudeHandler:
    def __init__(self):
        self.client = anthropic.Anthropic(api_key=ANTHROPIC_API_KEY)
        self.pdf_loader = PDFLoader()

    async def answer_text_question(self, question: str) -> str:
        """Matnli savolga javob berish"""
        # PDF dan kontekst olish
        pdf_context = self.pdf_loader.search_in_pdfs(question)

        if pdf_context:
            user_message = f"""Kitob matni:
{pdf_context}

```

```
Savol: {question}
```

Yuqoridagi kitob matniga asoslanib javob ber. Agar kitobda yo'q bo'lsa, o'z bilim

```
else:
```

```
    user_message = question
```

```
try:
```

```
    response = self.client.messages.create(  
        model="claude-opus-4-5",  
        max_tokens=1000,  
        system=SYSTEM_PROMPT,  
        messages=[{"role": "user", "content": user_message}]  
    )
```

```
    return response.content[0].text
```

```
except Exception as e:
```

```
    raise Exception(f"Claude API xatosi: {e}")
```

```
async def answer_image_question(self, image_url: str, question: str) -> str:
```

```
    """Rasmli savolga javob berish"""
```

```
try:
```

```
    # Rasmni yuklab olish
```

```
    async with httpx.AsyncClient() as client:
```

```
        resp = await client.get(image_url)
```

```
        image_data = base64.standard_b64encode(resp.content).decode("utf-
```

```
    # Rasm turini aniqlash
```

```
    content_type = resp.headers.get("content-type", "image/jpeg")
```

```
    if "png" in content_type:
```

```
        media_type = "image/png"
```

```
    elif "gif" in content_type:
```

```
        media_type = "image/gif"
```

```
    elif "webp" in content_type:
```

```
        media_type = "image/webp"
```

```
    else:
```

```
        media_type = "image/jpeg"
```

```
    response = self.client.messages.create(  
        model="claude-opus-4-5",  
        max_tokens=1000,  
        system=SYSTEM_PROMPT,  
        messages=[
```

```
            {
```

```
                "role": "user",
```

```
                "content": [
```

```
                    {
```

```
                        "type": "image",
```

```
        "source": {
            "type": "base64",
            "media_type": media_type,
            "data": image_data,
        },
    },
    {
        "type": "text",
        "text": question
    }
],
]
)
return response.content[0].text
except Exception as e:
    raise Exception(f"Rasm tahlilida xato: {e}")
```